

Lecture 6

Applying dropout to linear regression

Dropout setup

For feature i of training example j , let the dropout mask be

$$m_{ij} \sim \text{Bernoulli}(p).$$

Therefore,

$$E[m_{ij}] = p, \quad \text{Var}(m_{ij}) = p(1 - p).$$

Using inverted dropout,

$$\tilde{x}_{ij} = x_{ij} \frac{m_{ij}}{p}.$$

Hence

$$E[\tilde{x}_{ij}] = x_{ij}.$$

Applying dropout to linear regression

Prediction under dropout

For training example j , the corrupted input is the vector $\tilde{\mathbf{x}}_j$, and the random prediction is

$$z_j = \mathbf{w}^T \tilde{\mathbf{x}}_j = \sum_i w_i \tilde{x}_{ij}.$$

Substituting the dropout expression,

$$z_j = \sum_i w_i x_{ij} \frac{m_{ij}}{p}.$$

Now take expectation:

$$E[z_j] = \sum_i w_i x_{ij} E\left[\frac{m_{ij}}{p}\right].$$

Applying dropout to linear regression

$$E[z_j] = \sum_i w_i x_{ij} E\left[\frac{m_{ij}}{p}\right].$$

Since

$$E\left[\frac{m_{ij}}{p}\right] = 1,$$

we obtain

$$E[z_j] = \sum_i w_i x_{ij} = \mathbf{w}^T \mathbf{x}_j.$$

So dropout leaves the prediction unchanged **on average**.

Applying dropout to linear regression

Expected squared loss

For one training example, consider

$$E[(y_j - z_j)^2].$$

Using the bias–variance identity,

$$E[(y_j - z_j)^2] = (y_j - E[z_j])^2 + \text{Var}(z_j).$$

Since

$$E[z_j] = \mathbf{w}^T \mathbf{x}_j,$$

we get

$$E[(y_j - z_j)^2] = (y_j - \mathbf{w}^T \mathbf{x}_j)^2 + \text{Var}(z_j).$$

The new effect of dropout is therefore the variance term.

Applying dropout to linear regression

Variance introduced by dropout

Recall that

$$z_j = \sum_i w_i x_{ij} \frac{m_{ij}}{p}.$$

Because the masks are independent across features,

$$\text{Var}(z_j) = \sum_i w_i^2 x_{ij}^2 \text{Var}\left(\frac{m_{ij}}{p}\right).$$

But

$$\text{Var}\left(\frac{m_{ij}}{p}\right) = \frac{1}{p^2} p(1-p) = \frac{1-p}{p}.$$

Therefore,

$$\text{Var}(z_j) = \frac{1-p}{p} \sum_i w_i^2 x_{ij}^2.$$

Applying dropout to linear regression

Loss for one example

Substituting the variance back into the expected loss,

$$E[(y_j - \mathbf{w}^T \tilde{\mathbf{x}}_j)^2] = (y_j - \mathbf{w}^T \mathbf{x}_j)^2 + \frac{1-p}{p} \sum_i w_i^2 x_{ij}^2.$$

So the expected dropout loss is the ordinary squared error plus an additional quadratic penalty.

Applying dropout to linear regression

Sum over all training examples

Summing over j ,

$$E[\ell_{\text{drop}}(\mathbf{w})] = \sum_j (y_j - \mathbf{w}^T \mathbf{x}_j)^2 + \frac{1-p}{p} \sum_i w_i^2 \sum_j x_{ij}^2.$$

The ordinary least-squares part can be written as

$$\sum_j (y_j - \mathbf{w}^T \mathbf{x}_j)^2 = \|\mathbf{y}^T - \mathbf{w}^T X\|_2^2.$$

For each feature i ,

$$\sum_j x_{ij}^2 = \|\mathbf{x}_{i:}\|_2^2,$$

where $\mathbf{x}_{i:}$ contains feature i across all training examples.

Applying dropout to linear regression

Write the penalty in matrix form

Define the diagonal matrix

$$\Gamma = \text{diag} (\|\mathbf{x}_1\|_2, \dots, \|\mathbf{x}_d\|_2) .$$

Then

$$\|\Gamma \mathbf{w}\|_2^2 = \sum_i \|\mathbf{x}_i\|_2^2 w_i^2 .$$

Therefore the entire penalty can be written compactly as

$$\frac{1-p}{p} \|\Gamma \mathbf{w}\|_2^2 .$$

Applying dropout to linear regression

Final dropout objective

Putting the two parts together,

$$E[\ell_{\text{drop}}(\mathbf{w})] = \|\mathbf{y}^T - \mathbf{w}^T X\|_2^2 + \frac{1-p}{p} \|\Gamma \mathbf{w}\|_2^2.$$

Thus, with p defined as the **keep probability**, dropout is equivalent in expectation to ordinary least squares plus a **feature-dependent ridge penalty**.

Additional Regularization Techniques

- **L1 Regularization:** Encourages sparsity in the model parameters, often used for feature selection.
- **Mixup:** A data augmentation technique that generates new training examples by interpolating between pairs of examples and their labels.
- **Cutout:** A form of data augmentation where parts of the input data (usually images) are masked out.
- **Gradient Clipping:** Prevents exploding gradients by capping the gradients during backpropagation.

Batch Normalization

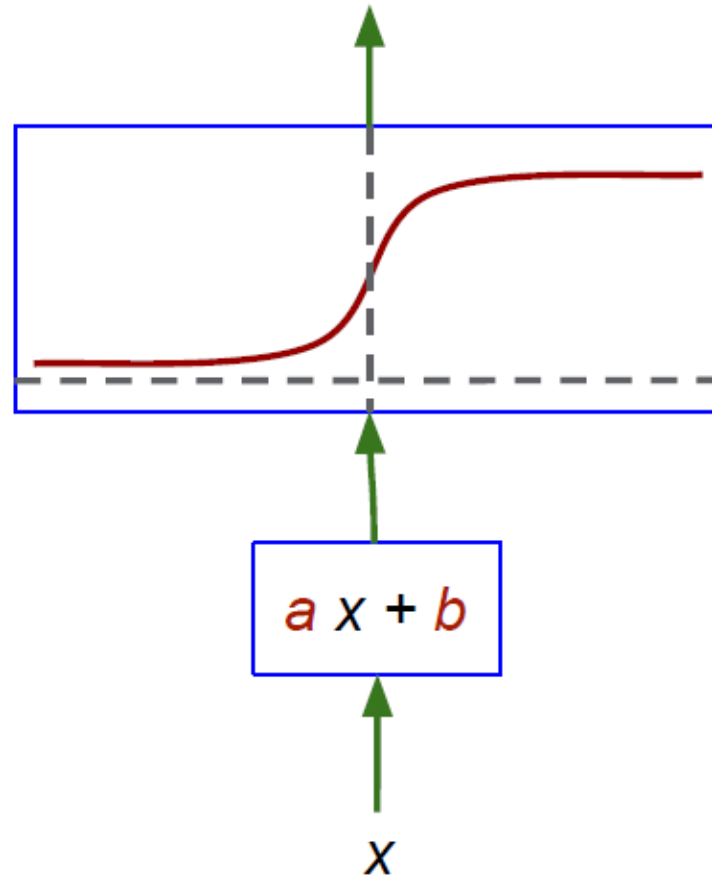
Slide modified from Sergey Ioffe , with permission

Slides based on

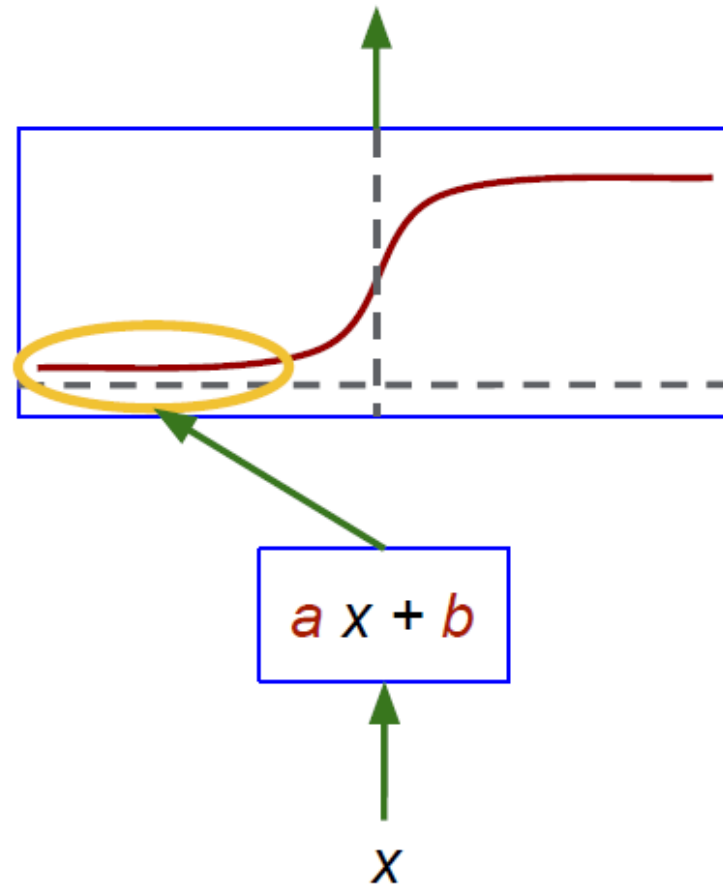
Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

By Sergey Ioffe and Christian Szegedy

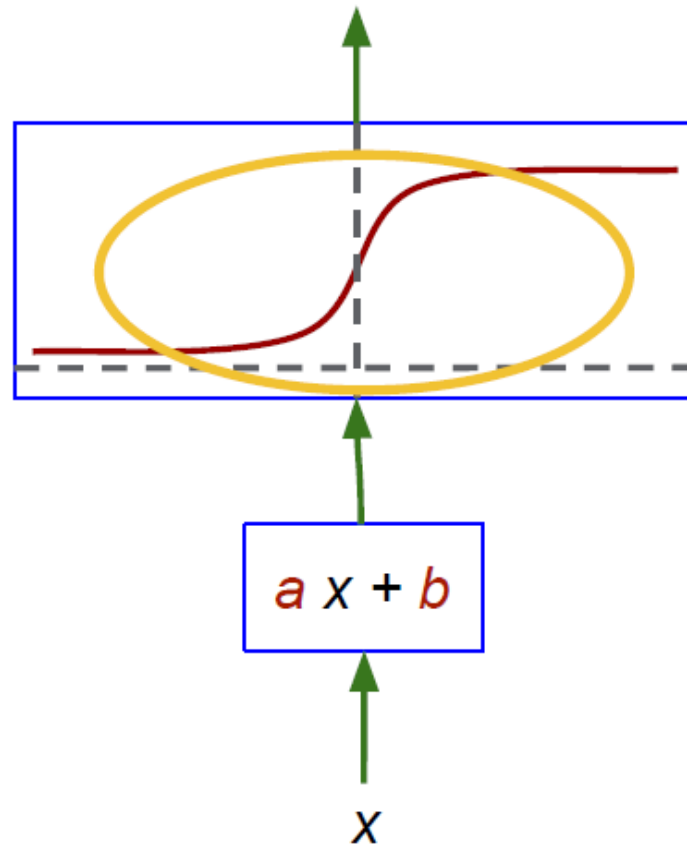
Batch Normalization



Batch Normalization

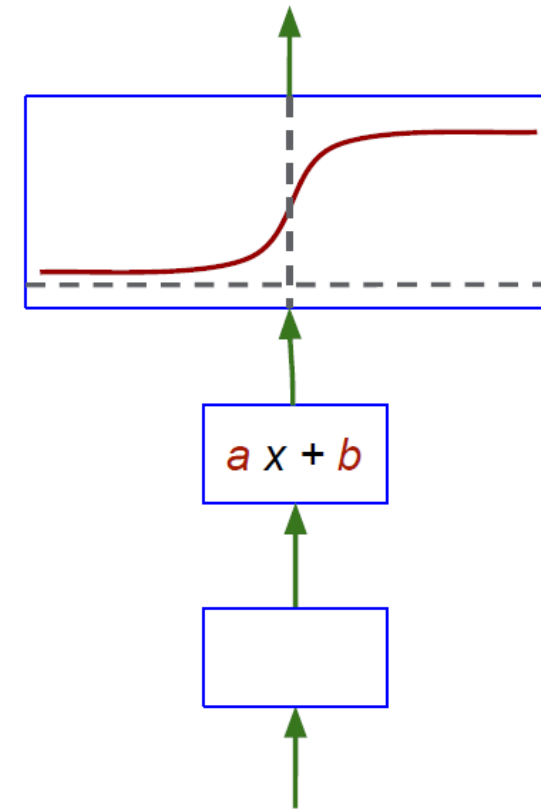


Batch Normalization



Effect of changing input distribution

- Careful initialization
- Small learning rates
- Rectifiers



Internal covariate shift

- Layer input distributions change during training

$$\ell = F_2(F_1(u, \Theta_1), \Theta_2)$$

- Change in internal activation distribution requires domain adaptation

- Normalize each activation:

$$x \mapsto \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x]}}$$

Mini-batch mean:

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

Mini-batch variance:

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

Normalize:

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

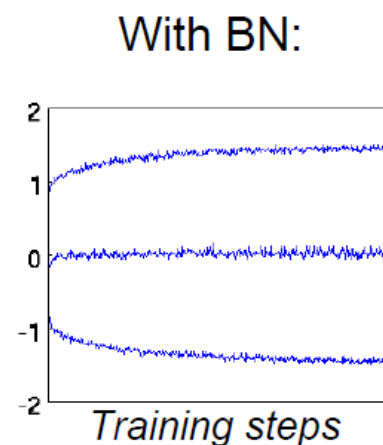
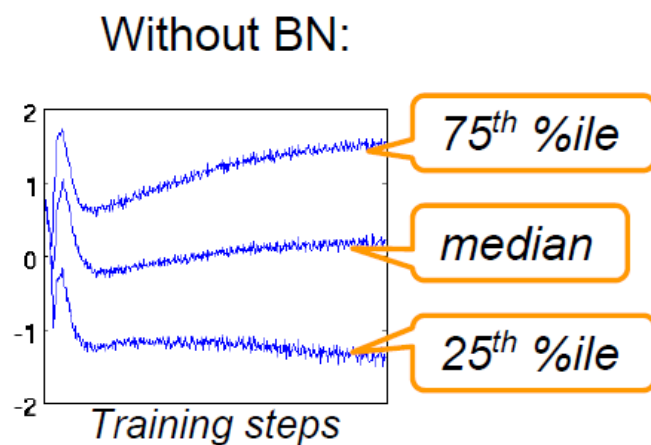
Scale and shift:

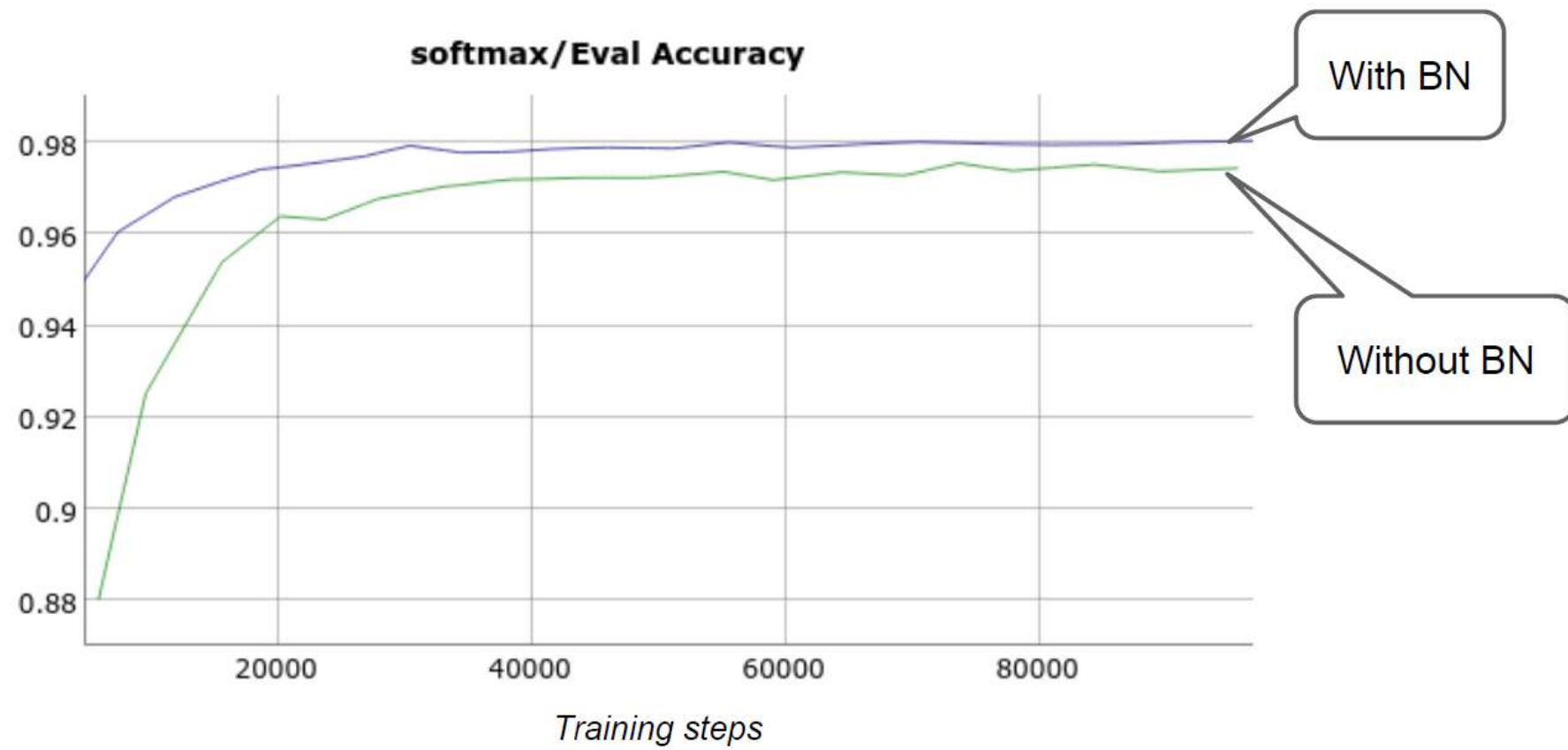
$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

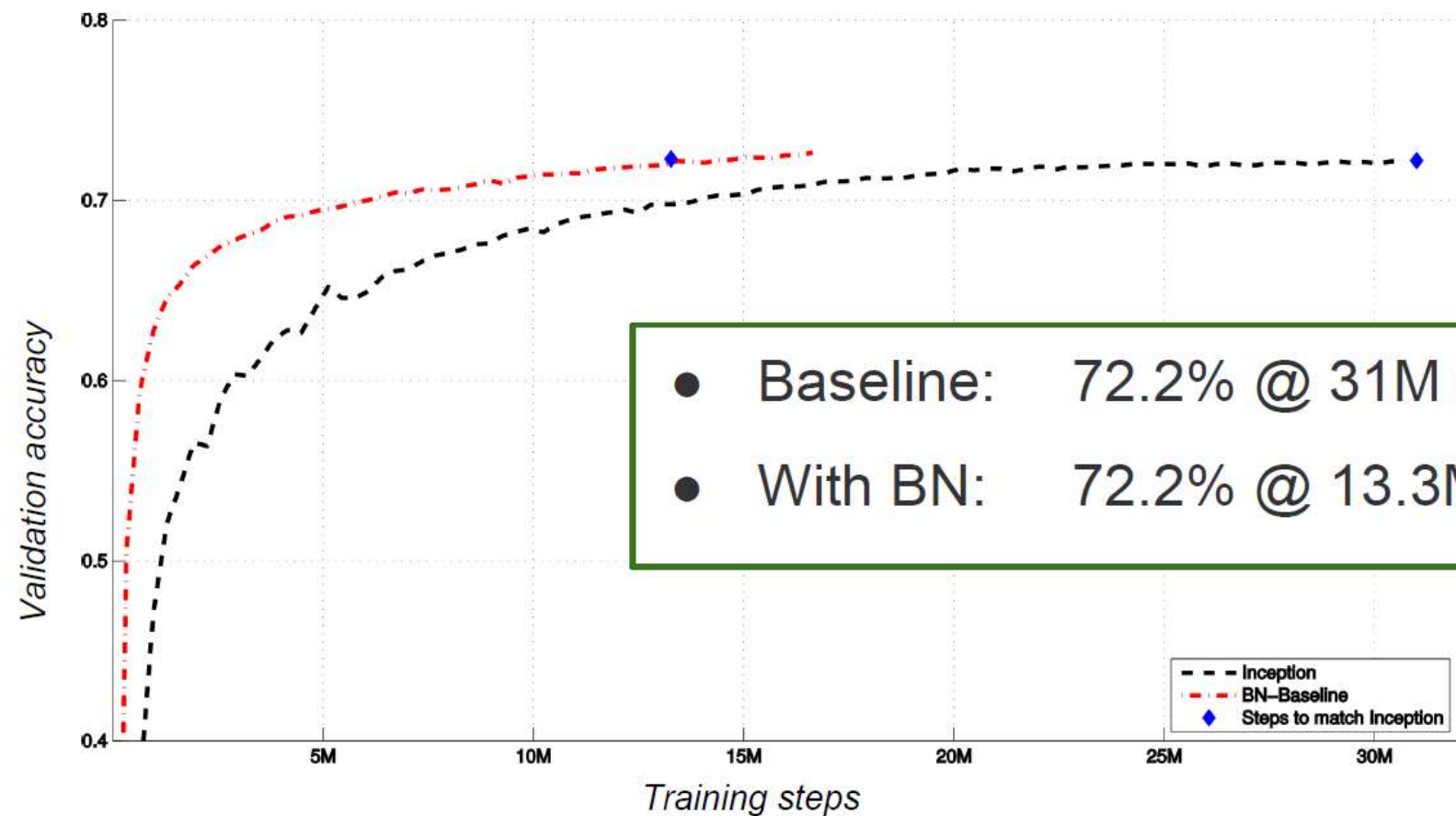
- Replace batch statistics with population statistics

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad \Rightarrow \quad \hat{x} \leftarrow \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}}$$

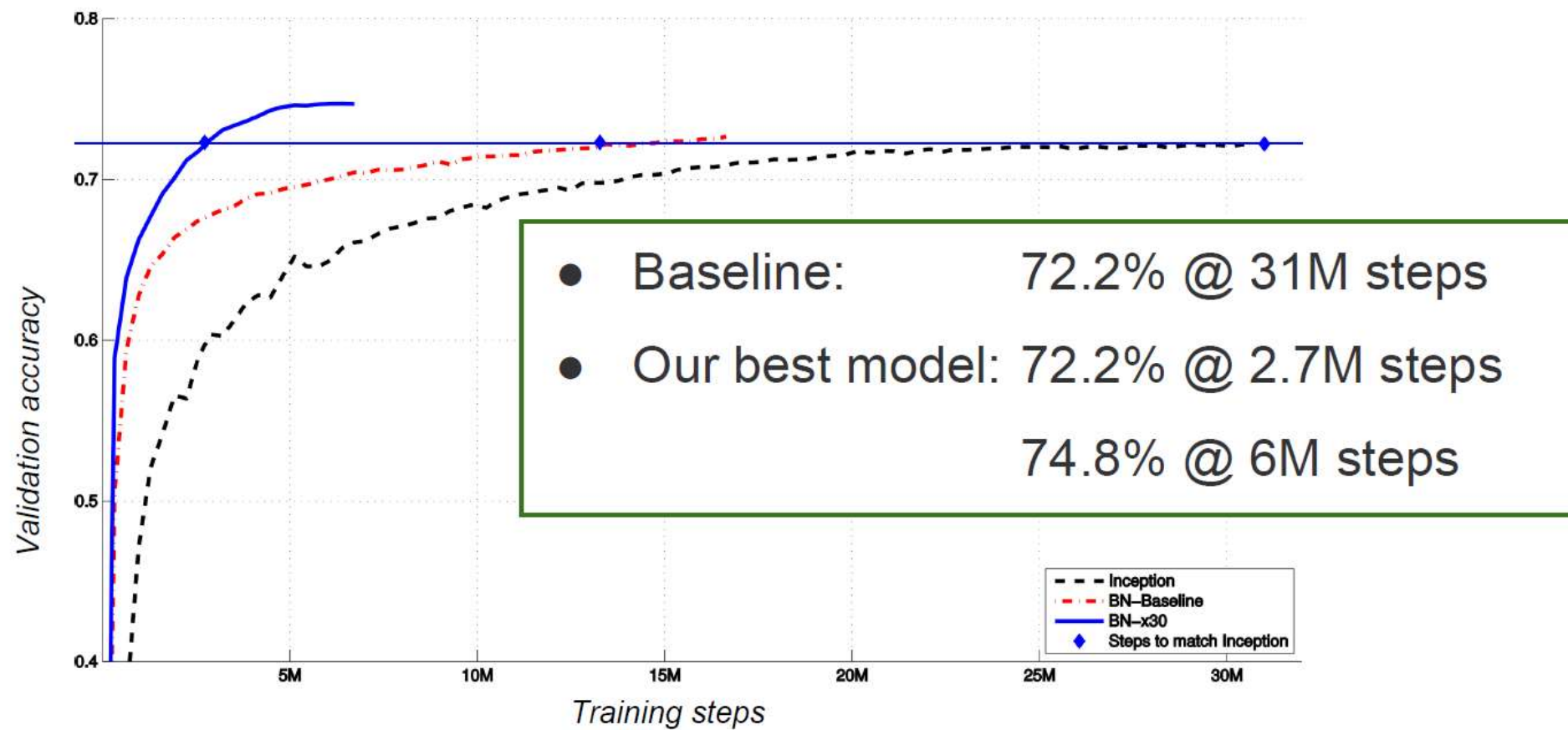
- MNIST: 3 FC layers + softmax, 100 logistic units per hidden layer
- Distribution of inputs to a typical sigmoid, evolving over 100k steps:







- Batch Normalization enables higher learning rate
 - Increased 30x
- Removing dropout improves validation accuracy
 - Batch Normalization as a regularizer?



Batch Normalization revised

BatchNorm

- Batch normalization motivated by internal covariate shift (2015 Ioffe & Szegedy)

BatchNorm

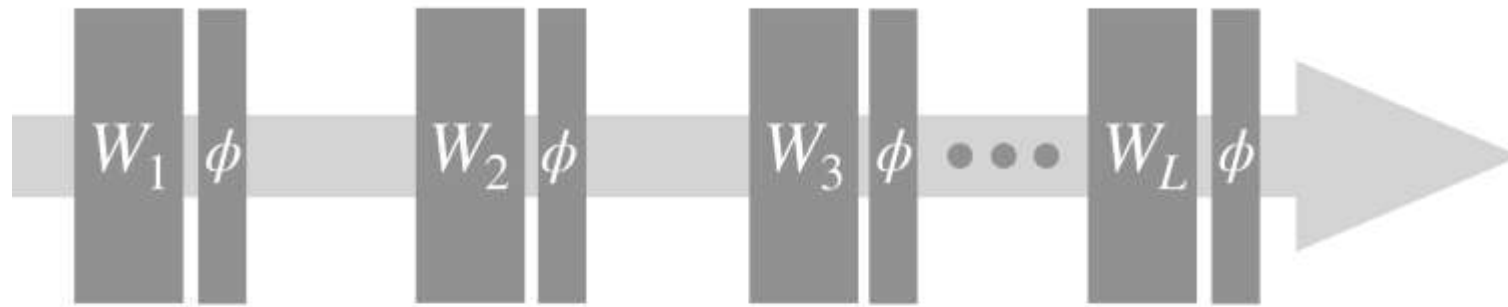
- Batch normalization motivated by internal covariate shift (2015 Ioffe & Szegedy)
- We have moving inputs.
- Distribution changes quite a lot.
- Solution: normalize the input of each layer.

BatchNorm

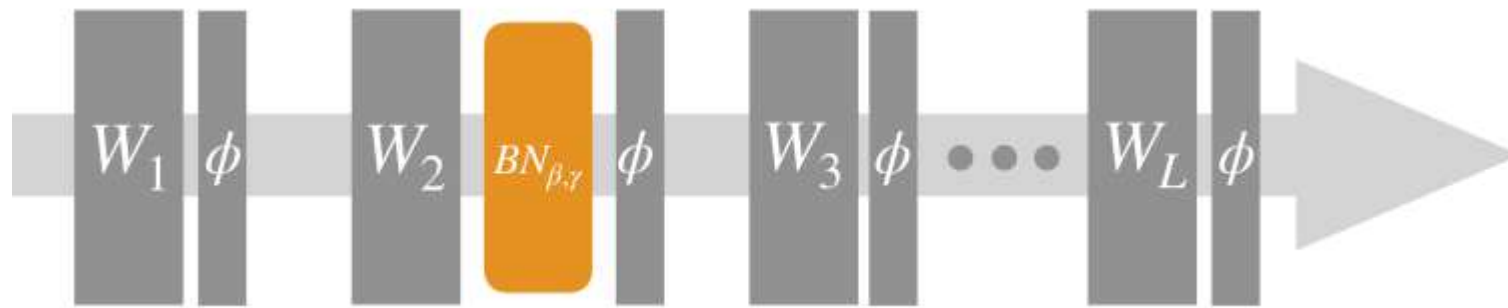
$$BN(y_j)^{(b)} = \gamma \left(\frac{y_j^{(b)} - \mu(y_j)}{\sigma(y_j)} \right) + \beta$$

- where $y_j^{(b)}$ denotes the value of the output y_j on the b -th input of a batch.
- and β and γ are learned parameters controlling the mean and variance of the output.

Standard Network

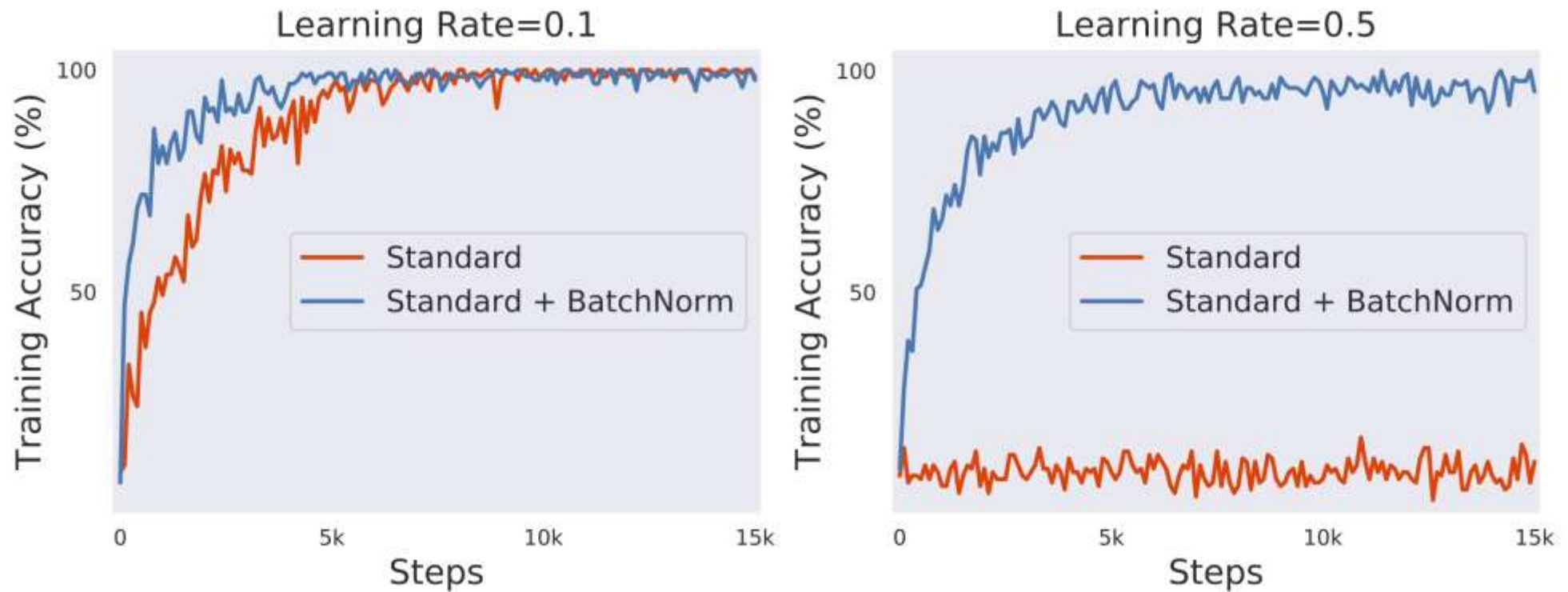


Adding a BatchNorm layer (between weights and activation function)



(Santurkar, et al., 2019)

Training with and without BatchNorm



(Santurkar, et al., 2019)

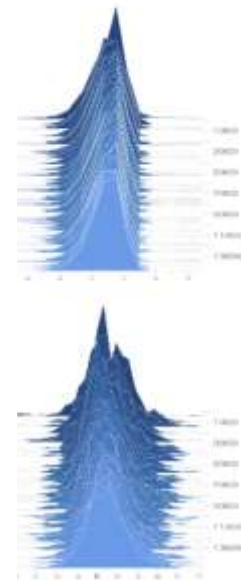
How Does Batch Normalization Help Optimization?

By Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (NeurIPS 2019).

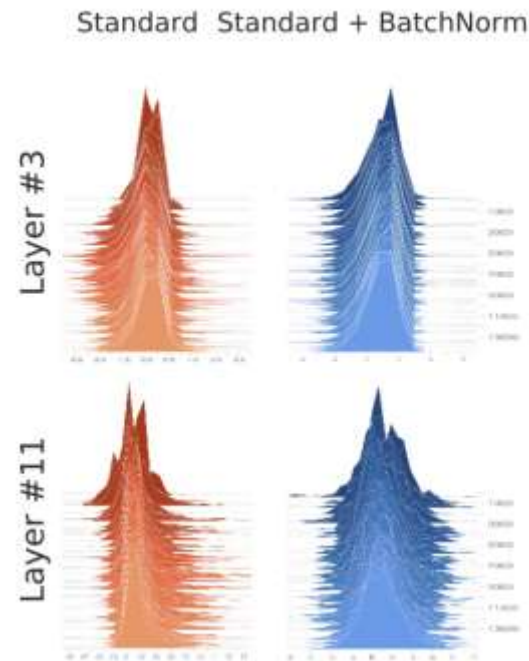
1. BatchNorm doesn't fix covariate shift.
2. If we fix covariate shift, it doesn't help.
3. If we intentionally increase ICS, it doesn't harm.
4. BatchNorm is not the only possible normalization. There are alternatives.

Training with and without BatchNorm

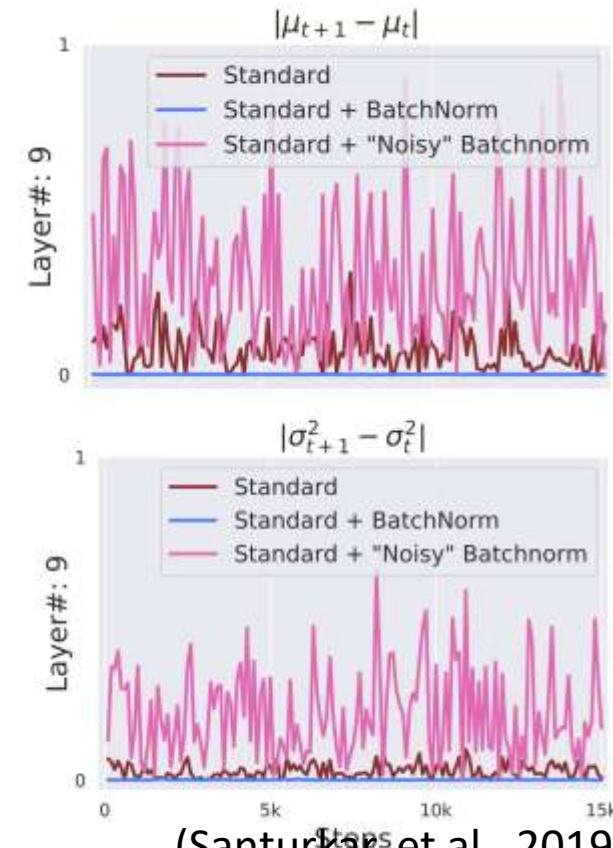
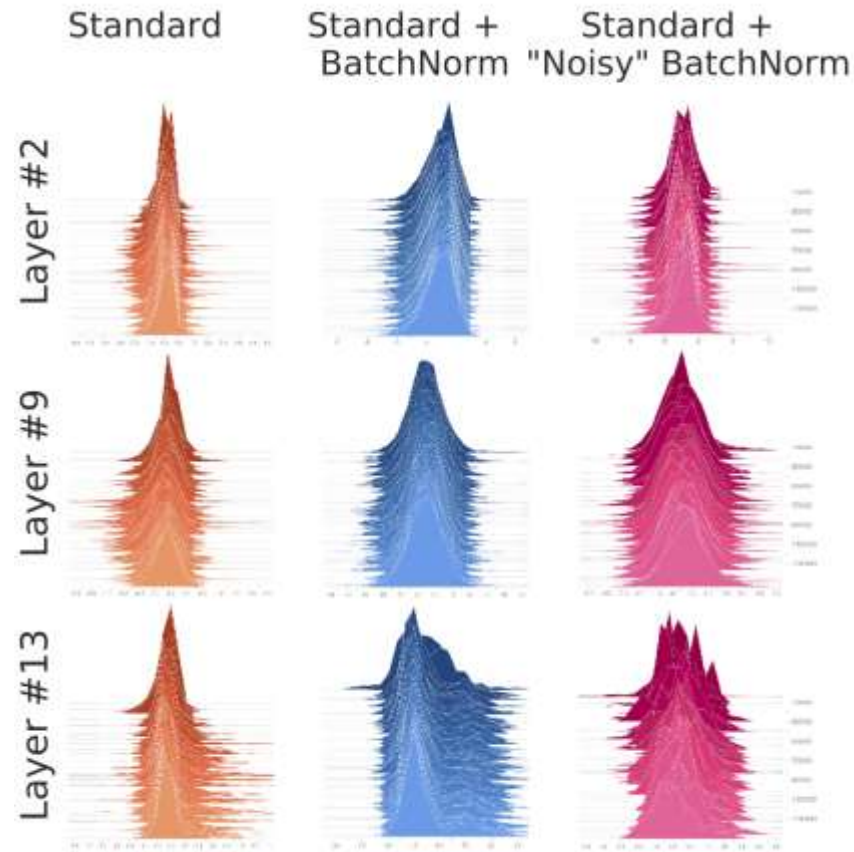
Standard + BatchNorm



Training with and without BatchNorm



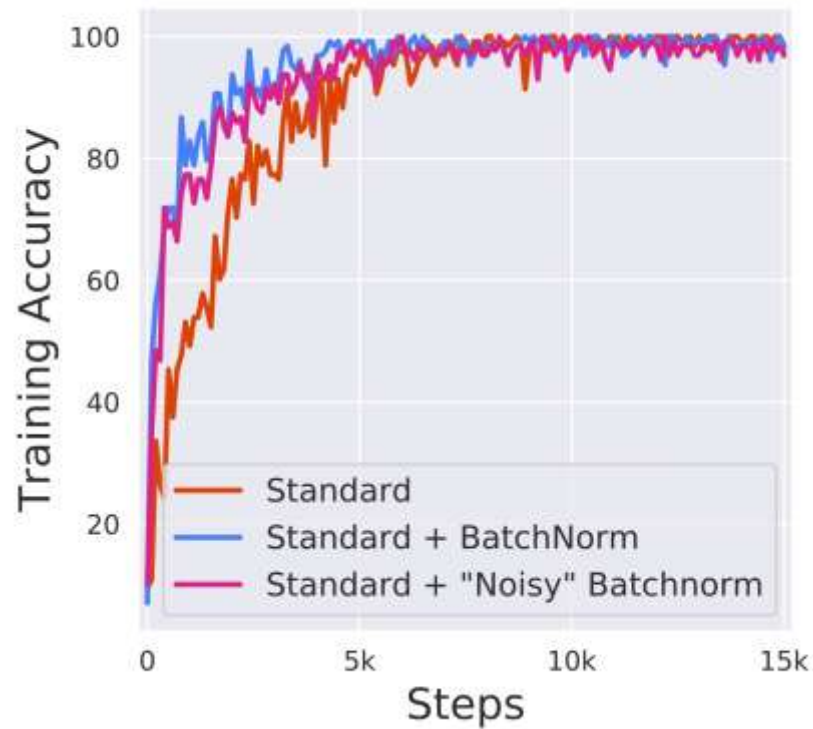
Increase Internal Covariate Shift



(Santurkar, et al., 2019)

Add non-stationary Gaussian noise (with a randomly sampled mean and variance at each iteration)

Increase Internal Covariate Shift



- The optimization performance is unaffected.
- A network with noisy BatchNorm converges faster
- Almost the same as BN

L –Lipschitz and β –smooth

f is L -Lipschitz if $\|f(x_1) - f(x_2)\| \leq L \|x_1 - x_2\|$, for all x_1 and x_2 .

f is β -smooth if its gradient is β -Lipschitz

BatchNorm's effect

- BatchNorm's reparameterization:
- Improves the Lipschitzness of the loss function.
the loss changes at a smaller rate and the magnitudes of the gradients are smaller.

BatchNorm's effect

- BatchNorm's reparameterization:
- Improves the Lipschitzness of the loss function.
the loss changes at a smaller rate and the magnitudes of the gradients are smaller.
- In gradient descent, we use the local linear approximation of the loss around the current solution to identify the best update step.
- How predictive of the nearby loss landscape this local approximation is

BatchNorm's effect

- BatchNorm's reparameterization:
- Improves the Lipschitzness of the loss function.
the loss changes at a smaller rate and the magnitudes of the gradients are smaller.

BatchNorm's effect

- BatchNorm's reparameterization:
- Improves the Lipschitzness of the loss function.
the loss changes at a smaller rate and the magnitudes of the gradients are smaller.
- Makes gradients of the loss more Lipschitz.
the loss exhibits a significantly better β -smoothness

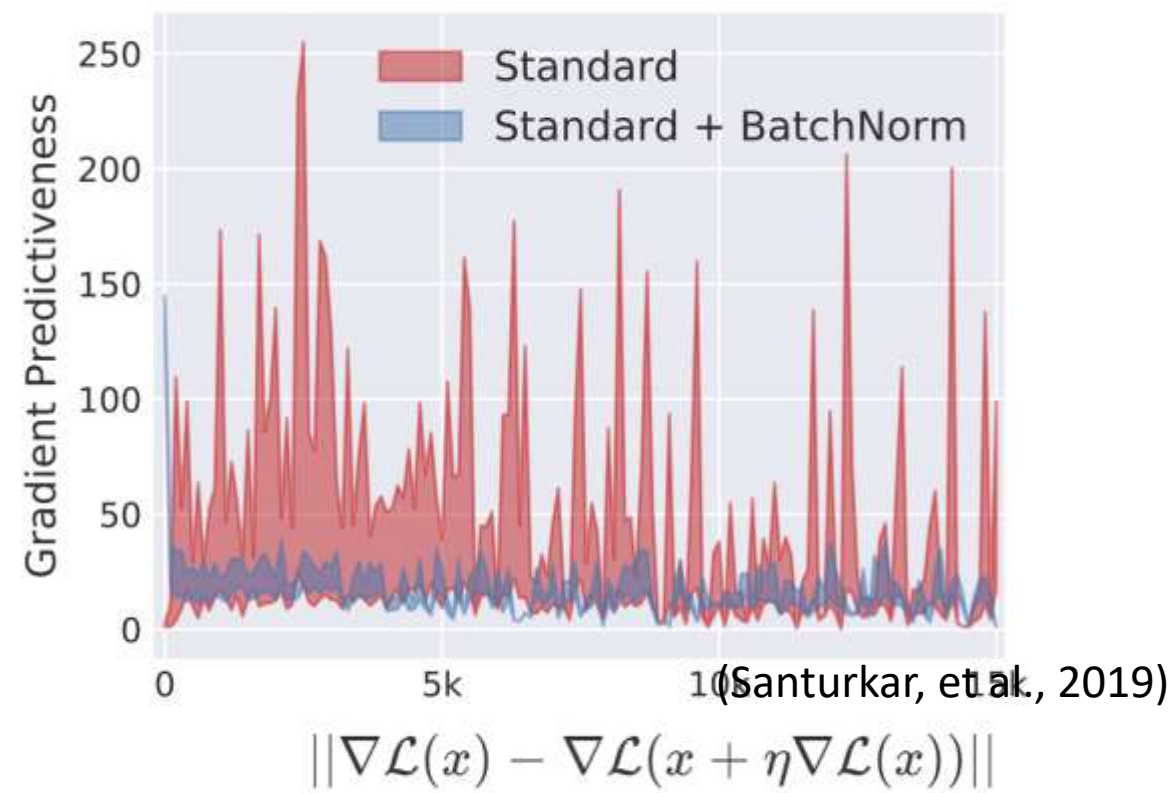
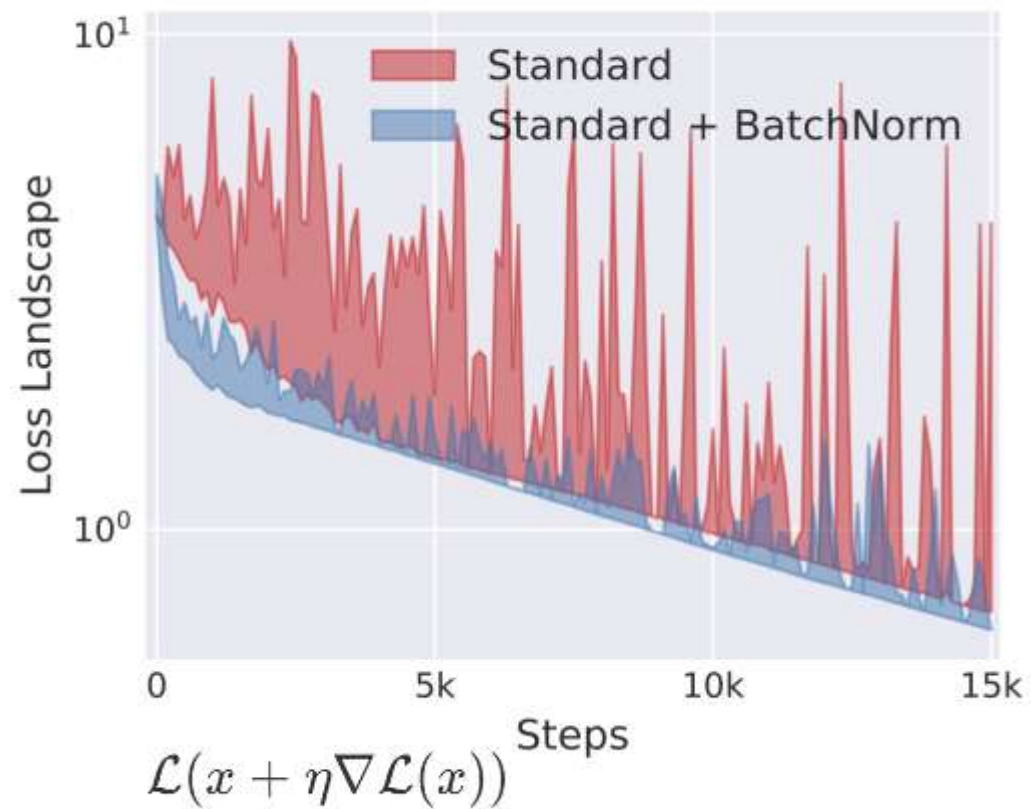
BatchNorm's effect

1. Variation of the value of the loss:

$$\mathcal{L}(x + \eta \nabla \mathcal{L}(x)), \quad \eta \in [0.05, 0.4]$$

2. Gradient predictiveness, i.e., the changes of the loss gradient:

$$\|\nabla \mathcal{L}(x) - \nabla \mathcal{L}(x + \eta \nabla \mathcal{L}(x))\|, \quad \eta \in [0.05, 0.4]$$



BatchNorm's effect

- A small variability of the loss indicates that the steps taken during training are unlikely to drive the loss uncontrollably high.
- A good gradient predictiveness implies that the gradient evaluated at a given point stays relevant over longer distances, hence allowing for larger step sizes.

How Does Batch Normalization Help Optimization?

By Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (NeurIPS 2019).

1. BatchNorm doesn't fix covariate shift.
2. If we fix covariate shift, it doesn't help.
3. If we intentionally increase ICS, it doesn't harm.
4. BatchNorm is not the only possible normalization. There are alternatives.

These effects are not unique to BatchNorm

- In this normalization

$$BN(y_j)^{(b)} = \gamma \left(\frac{y_j^{(b)} - \mu(y_j)}{\sigma(y_j)} \right) + \beta$$

replace denominator by l_p norm. i.e. $\|y\|_p$

Alternatives to BatchNorm

- l_p BatchNorm
- Normalization over layers

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. In arXiv preprint arXiv:1607.06450, 2016

- Subsets of the batch

Yuxin Wu and Kaiming He. Group normalization. In European Conference on Computer Vision (ECCV), 2018.

Alternatives to BatchNorm

- Weight Normalization (normalizing the weights instead of the activations)

Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In Advances in Neural Information Processing Systems (NIPS), 2016.

- ELU and SELU are two proposed non-linearities that have a decaying slope instead of a sharp saturation and can be used as an alternative for BatchNorm

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). In International Conference on Learning Representations (ICLR), 2016.

Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. In Advances in Neural Information Processing Systems (NIPS), 2017.